CODING · AULA 12 · 3 DE NOVEMBRO DE 2026

A vitrine ganha vida

JavaScript no navegador: a linguagem, o DOM e os eventos. Hoje a página deixa de ser um cartaz e passa a responder a quem está do outro lado.

Faculdade Senac Pernambuco · ADS 2026.2 · Prof. Sebastião Rogério

ONDE ESTAMOS NO SEMESTRE · 1 DE 3

O caminho até aqui

U1 · ✓ CONCLUÍDA

Fundamentos

11/08
Por que tantas linguagens

18/08
Paradigmas

25/08
Tipos e coleções

01/09
Funções, módulos e testes

U2 · ✓ CONCLUÍDA

Orientação a objetos

08/09
Classes

22/09
AV1 e encapsulamento

29/09
Herança e polimorfismo

06/10
Contratos e a Loja de Roupas

U3 · ✓ CONCLUÍDA

Back-end e API

13/10
HTTP, REST e a primeira API

20/10
Persistência e camadas

U4 · ◂ ESTAMOS AQUI

Front-end

27/10
Entrega 1, HTML e CSS

03/11
Hoje: JavaScript, DOM e eventos

10/11
Integração com a API

U5

Entrega e avaliação

17/11
PROJETO FINAL

24/11
AV2

01/12
Framework (sem nota)

08/12
Encerramento

ONDE ESTAMOS NO SEMESTRE · 2 DE 3

A aplicação que estamos construindo, camada por camada

Telas no navegador	HTML e CSS na Aula 11; hoje o comportamento · Aulas 11 a 13	◂ HOJE
▲ fetch (Aula 13) ▼		
API: HTTP + REST	a porta da loja para a rede · Aula 9	✓ pronto
▲ chama ▼		
Serviço e repositório	as camadas entre a porta e o banco · Aula 10	✓ pronto
▲ usa ▼		
Domínio e banco	as classes e o loja.db · Aulas 5 a 10	✓ pronto

- Na Aula 11 as telas nasceram **estáticas**: mostram o que está escrito no HTML e nada mais.
- Hoje elas passam a **reagir** — clique, digitação e envio — ainda com dados de exemplo na própria página.
- Na Aula 13 elas vão **buscar os dados na API** e a aplicação fecha de ponta a ponta.

ONDE ESTAMOS NO SEMESTRE · 3 DE 3

O que falta: as notas fecham em 27/11

DATA	AULA	TEMA	O QUE ACRESCENTA AO PROJETO
27/10	11	Entrega 1 · HTML e CSS	a API foi avaliada e nasceu a primeira tela
03/11	**12**	**JavaScript, DOM e eventos**	**hoje: as telas reagem ao usuário**
10/11	13	Assincronismo e integração	as telas conversam com a API
17/11	14	**PROJETO FINAL**	apresentação com demonstração e a documentação: 50% do segundo processo
24/11	15	**AV2** e devolutiva	prova: 50% do segundo processo
01/12 e 08/12	16 e 17	Framework e encerramento	conteúdo complementar, sem nota

O QUE VAMOS VER HOJE

A terceira linguagem do navegador

ABERTURA

De onde viemos

`retomada da Aula 11`

- A vitrine que vocês fizeram
- O botão que não faz nada

MUNDO 1

A linguagem

`teoria com exemplos`

- A terceira linguagem
- Valores e funções
- Listas e objetos

MUNDO 2

DOM e eventos

`teoria com exemplos`

- A página é objeto
- Mudar a página
- Eventos

MUNDO 3

Prática na loja

`vocês seguem a aula`

- O primeiro script
- A vitrine filtra
- O formulário reage

ABERTURA · RETOMADA DA AULA 11

Vocês fizeram um botão. Por que ele não faz nada?

front/cadastro.html

```
<form>
  <label for="nome">Nome do produto</label>
  <input id="nome" name="nome" required>
  <button>Cadastrar</button>
</form>
```

- O **HTML** criou o botão: ele existe na página.
- O **CSS** deixou o botão apresentável: cor, tamanho, espaçamento.
- Ninguém disse ao botão **o que fazer** — e é exatamente essa frase que falta no projeto de vocês.
- Essa frase se escreve em **JavaScript**, e é a aula de hoje.
- **Não precisa anotar sintaxe.** Tudo do mundo 1 está na trilha, que vocês recebem no fim da aula. Hoje o que importa é entender **onde o JavaScript difere do Python**.



O MAPA DE HOJE

Nove fases, três mundos

Mundos 1 e 2: a linguagem, o DOM e os eventos, com exemplos. Mundo 3: a vitrine da Loja de Roupas ganhando comportamento, fase por fase, com checkpoints.



FASE 1-1 · CONTEÚDO NOVO

A terceira linguagem

Onde o JavaScript roda, e por que ele é diferente do Python que vocês já escrevem.

CONCEITO AULA 12

Onde o JavaScript roda

O Python de vocês roda **no servidor**: a requisição chega, o FastAPI responde, e o usuário nunca vê esse código. O JavaScript roda **no navegador** da pessoa, na máquina dela, depois que a página chegou. É a única linguagem que o navegador executa.

exemplo

```
servidor (Python)          navegador (JavaScript)
-----------------          ----------------------
FastAPI, SQLAlchemy        o que acontece na tela
as regras do negócio       clique, digitação, envio
o banco de dados           o que aparece e some

front/app.js  <- o arquivo de hoje
```

CUIDADO

Todo JavaScript que vocês escrevem **chega inteiro ao navegador** e pode ser lido por qualquer um com dois cliques. Senha, token e regra de permissão ficam no servidor, sempre.

CONCEITO AULA 12

O console: onde o JavaScript fala com você

O Python avisa no terminal; o JavaScript avisa no **console do navegador**. Abre com **F12** (ou Ctrl+Shift+I) na aba *Console*. É lá que aparece o que o seu `console.log` imprime e, em vermelho, todo erro do seu código. A partir de hoje ele fica aberto o tempo todo, ao lado da página.

console do navegador

```
> console.log("oi")            // você manda imprimir
oi

> 2 + 2                        // dá para testar na hora
4

> document.querySelector("h1")
<h1>Vitrine</h1>

✗ Uncaught TypeError: ... is null   <- vermelho = bug seu
```

PENSE ASSIM

O console é o `print()` do navegador, e também um Python interativo: dá para digitar uma linha e ver o resultado sem salvar arquivo nenhum.

CONCEITO AULA 12

let e const

Em Python você atribui e pronto. Em JavaScript toda variável nasce com uma palavra na frente, e essa palavra decide se ela pode ser **reatribuída**. Regra prática: escreva tudo com `const`; quando o navegador reclamar, troque aquela linha para `let`.

exemplo

```
# Python                    // JavaScript
nome = "Ana"                let nome = "Ana";
nome = "Bia"                nome = "Bia";     // ok

PI = 3.14                   const PI = 3.14;
PI = 3.15   # permitido     PI = 3.15;        // TypeError
```

CUIDADO

`const` trava a variável, não o conteúdo. Com `const lista = [1, 2]`, a linha `lista.push(3)` funciona; o que dá erro é `lista = [4]`.

FASE 1-1 · DÁ ERRO OU NÃO?

Diga o que acontece em cada linha, e por quê.

- a) `const n = 10; n = 11;`
- b) `const aluno = { nome: "Ana" };`
  `aluno.nome = "Bia";`
- c) `let x; x = 5;`
- d) `const lista = []; lista.push("a");`

Ver resposta

a) erro. Reatribuir a variável é o que `const` proíbe.

b) funciona. O objeto continua o mesmo; só um campo dentro dele mudou.

c) funciona. `let` permite declarar agora e atribuir depois.

d) funciona. Mesmo caso do b: a lista é a mesma, só ganhou um item.



FASE 1-2 · CONTEÚDO NOVO

Valores e funções

As comparações que enganam e a forma curta de escrever função.

CONCEITO AULA 12

Igualdade: use sempre ===

JavaScript tem **dois** operadores de igualdade. O `==` converte os tipos antes de comparar e produz resultados que ninguém espera. O `===` compara tipo e valor, e é o que corresponde ao `==` do Python.

exemplo

```
1 ==  "1"      // true   <- converteu o texto em número
1 === "1"      // false  <- use este

0 == ""        // true
null == undefined  // true

null       // vazio de propósito
undefined  // nunca recebeu valor
```

CUIDADO

Não existe caso nesta disciplina em que `==` seja a escolha certa. Use `===` e `!==` sempre.

CONCEITO AULA 12

Os seis valores falsos

Em Python, lista vazia e dicionário vazio são falsos. **Em JavaScript não.** Os valores falsos são exatamente seis: `false`, `0`, `""`, `null`, `undefined` e `NaN` (*Not a Number*, "não é um número": o que sobra de uma conta inválida, como `Number("abc")`). Todo o resto é verdadeiro.

exemplo

```
if ([]) {
  // ENTRA AQUI. em Python, não entraria
}

const produtos = [];

if (!produtos)              { }  // nunca executa
if (produtos.length === 0)  { }  // o jeito certo
```

CUIDADO

Para saber se uma lista está vazia, teste `lista.length === 0`. O `if (not lista)` que funciona em Python vira bug silencioso aqui.

CONCEITO AULA 12

Função, função de seta e texto com variável

Três novidades de uma vez. A *arrow function* (função de seta, `=>`) é a forma curta e sem chaves ela já devolve o resultado. A *template string* (texto de modelo), entre crases com `${ }`, faz o papel da f-string. E **função é valor**: dá para guardar numa variável e passar como argumento — o mundo 2 inteiro depende disso.

exemplo

```
# Python                          // JavaScript
def soma(a, b):                   function soma(a, b) {
    return a + b                    return a + b;
                                  }

dobro = lambda x: x * 2           const dobro = (x) => x * 2;

f"R$ {p:.2f}"                     `R$ ${p.toFixed(2)}`
```

CUIDADO

`const dobro = x => { x * 2 }` devolve `undefined`. Assim que aparecem as chaves, o `return` volta a ser obrigatório.

O `if` entra ou não entra?

- a) `if ("0")`
- b) `if (0)`
- c) `if ({})`
- d) `if (produtos.length)`, com a lista vazia

Ver resposta

a) entra. É um texto com um caractere; só o texto vazio é falso.

b) não entra. O número zero é um dos seis falsos.

c) entra. Objeto vazio é verdadeiro — ao contrário do Python.

d) não entra. `length` vale `0`, que é falso. Funciona, mas prefira `=== 0`: diz o que você quer dizer.



FASE 1-3 · CONTEÚDO NOVO

Listas e objetos

O que corresponde à lista e ao dicionário do Python, e como percorrer sem escrever laço.

CONCEITO AULA 12

Array e objeto

A lista do Python é o *array* (vetor); o dicionário é o *object* (objeto), com a chave escrita **sem aspas**. Os dois se combinam igual ao Python: array de objetos, objeto com array dentro. É assim que a vitrine vai guardar os produtos.

exemplo

```
# Python                        // JavaScript
len(x)                          x.length
x.append(4)                     x.push(4)
d["nome"]                       d.nome
for p in produtos:              for (const p of produtos)
d.get("x")                      d.x   // undefined, sem erro

const produtos = [
  { nome: "Camiseta", preco: 39.9, tam: "M" },
  { nome: "Calça",    preco: 129.0, tam: "G" }
];
```

CUIDADO

Ler uma chave que não existe devolve `undefined` em vez de erro. `produto.prceo` não quebra nada — e o problema só aparece três linhas depois.

CONCEITO AULA 12

map, filter e sort

Onde o Python usa compreensão de lista, o JavaScript chama um **método passando uma função**. `map` transforma cada item, `filter` seleciona, `sort` ordena. É o uso prático de "função é valor".

exemplo

```
# Python
nomes = [p["nome"] for p in produtos]
baratos = [p for p in produtos if p["preco"] < 50]

// JavaScript
const nomes   = produtos.map(p => p.nome);
const baratos = produtos.filter(p => p.preco < 50);
const porPreco = [...produtos].sort((a, b) => a.preco - b.preco);
```

CUIDADO

`[10, 9, 1].sort()` devolve `[1, 10, 9]`: sem função de comparação, `sort` ordena como texto. Para números, `.sort((a, b) => a - b)`.

Como fica em JavaScript?

- a) `[p["nome"] for p in produtos]`
- b) `[p for p in produtos if p["tam"] == "M"]`
- c) `len(produtos)`
- d) ordenar os produtos do mais barato ao mais caro

Ver resposta

a) `produtos.map(p => p.nome)`

b) `produtos.filter(p => p.tam === "M")`

c) `produtos.length`

d) `[...produtos].sort((a, b) => a.preco - b.preco)` — a cópia evita bagunçar a lista original.



FASE 2-1 · CONTEÚDO NOVO

A página é objeto

O navegador transforma o seu HTML numa árvore que o programa consegue ler e alterar.

CONCEITO AULA 12

DOM: a página virou árvore de objetos

DOM é *Document Object Model* — Modelo de Objetos do Documento. É a tradução do HTML que vocês escreveram na Aula 11 numa árvore de objetos viva na memória do navegador. **Mudar o objeto muda a tela na hora**; o arquivo no disco continua igual.

exemplo

```
<main>                              document
  <h1>Vitrine</h1>                    └ main
  <section id="vitrine">                ├ h1   "Vitrine"
    <article class="produto">           └ section#vitrine
      ...                                     └ article.produto
    </article>
  </section>
</main>
```

PENSE ASSIM

O HTML é a planta da casa; o DOM é a casa construída. Você não reescreve a planta para mover um móvel — mexe na casa. Recarregar a página reconstrói tudo a partir da planta.

Do arquivo ao comportamento

#	O QUE ACONTECE	QUEM FAZ
1	O navegador baixa o `index.html`	rede
2	Lê o HTML de cima para baixo e monta a árvore do DOM	navegador
3	Aplica o `estilo.css` a cada nó da árvore	navegador
4	Encontra o `<script src="app.js">` e executa o arquivo	navegador
5	**O seu código encontra elementos, altera e registra o que fazer nos eventos**	**vocês**
6	A pessoa clica, digita, envia — e o navegador chama as suas funções	usuário

- O passo 4 vem **depois** do 2: por isso o `<script>` vai no fim do `<body>`. No `<head>`, ele rodaria antes de os elementos existirem.



FASE 2-2 · CONTEÚDO NOVO

Mudar a página

Encontrar um elemento, trocar o conteúdo dele e criar elementos novos a partir de dados.

CONCEITO AULA 12

Encontrar: querySelector

Dois métodos resolvem quase tudo, e a sintaxe de busca é **a mesma do seletor de CSS** que vocês aprenderam na Aula 11. `querySelector` devolve um elemento — o primeiro que casar — ou `null`. `querySelectorAll` devolve todos, numa lista.

front/app.js

```
const titulo  = document.querySelector("h1");
const vitrine = document.querySelector("#vitrine");
const cards   = document.querySelectorAll(".produto");

console.log(cards.length);   // quantos cartões existem
cards.forEach(c => console.log(c.textContent));
```

PENSE ASSIM

É a busca do CSS, com a mesma escrita: `#id`, `.classe`, `tag`. O que muda é o que você faz depois — lá você pintava, aqui você programa.

CONCEITO AULA 12

Alterar e criar

`textContent` troca o texto. `classList` liga e desliga classes do CSS — ou seja, o JavaScript **aciona o estilo** que já existe, em vez de escrever cor na mão. E `createElement` monta um pedaço de página a partir de dados.

front/app.js

```
titulo.textContent = "Vitrine da Loja";
card.classList.add("esgotado");      // liga a classe do CSS
card.classList.remove("oculto");

function montarCard(p) {
  const art = document.createElement("article");
  art.className = "produto";
  art.innerHTML = `<h3>${p.nome}</h3><p>R$ ${p.preco.toFixed(2)}</p>`;
  return art;
}

vitrine.innerHTML = "";              // limpa antes de desenhar
produtos.forEach(p => vitrine.append(montarCard(p)));
```

CUIDADO

Nunca monte `innerHTML` com algo que o usuário digitou: é a porta de entrada de código estranho na sua página. E limpar antes de desenhar evita a lista duplicada.

O código abaixo imprime `null` e depois quebra. Onde está o erro?

- `<head>`
- `<script src="app.js"></script>`
- `</head>` … `<section id="vitrine">`
- e em `app.js`:
  `document.querySelector("#vitrine").innerHTML = "";`

Ver resposta

O script rodou antes de o elemento existir.

O navegador lê a página de cima para baixo: no `<head>`, a `section` ainda não foi criada.

Correção: mover o `<script>` para o fim do `<body>`.

Alternativa: manter no `head` com o atributo `defer`, que adia a execução.



FASE 2-3 · CONTEÚDO NOVO

Eventos

O que faz um botão deixar de ser enfeite.

CONCEITO AULA 12

addEventListener

Um evento é algo que aconteceu na página: um clique, uma tecla, um formulário enviado. `addEventListener` (ouvinte de evento) recebe **o nome do evento e uma função**. A função não roda agora: fica guardada, e o navegador a chama quando o evento acontecer. É a *callback* (função de retorno).

front/app.js

```
const botao = document.querySelector("#filtrar");

botao.addEventListener("click", function () {
  console.log("alguém clicou");
});

// mesma coisa, com função de seta
botao.addEventListener("click", () => console.log("clicou"));
```

CUIDADO

`addEventListener("click", minhaFuncao())` — com parênteses — executa a função na hora e registra o **resultado** dela. Passe o nome **sem** parênteses.

CONCEITO AULA 12

click, input, submit — e o preventDefault

Com três eventos dá para fazer o front-end inteiro do projeto. A função recebe um objeto com os dados do evento, por convenção chamado `e`: `e.target` é o elemento onde aconteceu e `e.target.value` é o que foi digitado.

front/app.js

```
campo.addEventListener("input", (e) => {
  console.log("digitou:", e.target.value);
});

form.addEventListener("submit", (e) => {
  e.preventDefault();      // sem isto, a página recarrega
  const nome = document.querySelector("#nome").value;
  console.log({ nome });
});
```

CUIDADO

O comportamento padrão do formulário é **recarregar a página**, de um tempo anterior ao JavaScript. `preventDefault` diz "eu cuido disso" — sem ele, o seu código some antes de terminar.

Escolha o evento adequado para cada caso.

- a) Filtrar a vitrine enquanto a pessoa digita.
- b) Abrir os detalhes ao tocar no cartão do produto.
- c) Validar os campos antes de cadastrar um produto.
- d) Avisar que o campo de preço ficou vazio, só ao sair dele.

Ver resposta

a) input: dispara a cada tecla.

b) click: serve para toque no celular também.

c) submit no `<form>`, com `preventDefault` — e não `click` no botão, que deixaria o Enter de fora.

d) blur: dispara quando o campo perde o foco. Existem muitos outros; estes quatro cobrem o projeto.

HORA DA PRÁTICA

Agora é com vocês

- Abram a pasta `loja-roupas` no editor, a mesma da Aula 11.
- Três fases, cada uma com um checkpoint: salvar, rodar o `pytest` e subir.
- Hoje os produtos ainda ficam **escritos no próprio app.js**. Na Aula 13 eles vêm da API.
- Travou numa fase? O slide do checkpoint tem o link da fase pronta.
- **Deixem o console aberto** (F12) ao lado do editor: é ele que diz o que deu errado.
- Esqueceu alguma coisa do mundo 1? A **trilha** refaz tudo em sete aulas curtas — consultem durante a prática, não decorem.



FASE 3-1 · PRÁTICA

O primeiro script

Ligar o arquivo na página e desenhar a vitrine a partir de uma lista.

FASE 3-1 · NO EDITOR

front/index.html: ligar o script

front/index.html

```
  <section id="vitrine"></section>

  <script src="app.js"></script>
</body>
</html>
```

- A `section` agora nasce **vazia**: quem põe os cartões é o JavaScript.
- O `<script>` fica na **última linha** antes do `</body>`.
- Os cartões que vocês escreveram na Aula 11 saem do HTML — guardem um deles de modelo.

FASE 3-1 · NO EDITOR

front/app.js: a vitrine desenhada por código

front/app.js

```
const produtos = [
  { nome: "Camiseta básica", preco: 39.9,  tam: "M" },
  { nome: "Calça jeans",     preco: 129.0, tam: "G" },
  { nome: "Moletom",         preco: 159.9, tam: "M" }
];

const vitrine = document.querySelector("#vitrine");

function montarCard(p) {
  const art = document.createElement("article");
  art.className = "produto";
  art.innerHTML = `<h3>${p.nome}</h3>
                   <p>R$ ${p.preco.toFixed(2)}</p>
                   <p>Tamanho ${p.tam}</p>`;
  return art;
}

function desenhar(lista) {
  vitrine.innerHTML = "";
  lista.forEach(p => vitrine.append(montarCard(p)));
}

desenhar(produtos);
```

- A lista de produtos é o **dado**; `desenhar` é a **tela**. Separados desde já.
- `toFixed(2)` formata o preço com duas casas — é o `:.2f` do Python.
- `desenhar` sempre limpa antes: chamar de novo não duplica.
- O CSS da Aula 11 continua valendo: a classe `produto` é a mesma.

FASE 3-1 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
tudo verde
$ git add .
$ git commit -m "Fase 3-1: vitrine desenhada por JavaScript"
$ git push
```

CONFIRA

A vitrine abre com os três produtos e o **visual da Aula 11 intacto**. No console do navegador (F12), nenhum erro em vermelho.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-19.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-19`



FASE 3-2 · PRÁTICA

A vitrine filtra

Um campo de busca que filtra os produtos enquanto a pessoa digita.

FASE 3-2 · NO EDITOR

front/app.js: filtrar enquanto digita

front/app.js

```
// no index.html, dentro do header:
//   <input id="busca" type="search" placeholder="Buscar produto">

const busca = document.querySelector("#busca");

busca.addEventListener("input", (e) => {
  const termo = e.target.value.toLowerCase();

  const achados = produtos.filter(
    p => p.nome.toLowerCase().includes(termo)
  );

  desenhar(achados);
});
```

- O evento `input` dispara a cada tecla: o filtro acontece enquanto a pessoa digita.
- `toLowerCase` dos dois lados faz a busca ignorar maiúsculas.
- `filter` devolve uma lista nova; `produtos` continua intacta — por isso dá para apagar a busca e tudo voltar.
- Repare: **a função desenhar não mudou**. Dado e tela separados pagam dividendo já na segunda fase.

FASE 3-2 · NO NAVEGADOR

Conferir como um profissional

#	O QUE FAZER	O QUE TEM QUE ACONTECER
1	Digitar `cam` no campo de busca	só a camiseta fica na vitrine
2	Apagar tudo	os três produtos voltam
3	Digitar `CAM`, em maiúsculas	mesmo resultado do passo 1
4	Digitar `zzz`	vitrine vazia, e **nenhum erro** no console
5	Abrir o F12 na aba Console	limpo: vermelho ali é bug seu

- O passo 4 costuma revelar o cartão duplicado de quem esqueceu o `vitrine.innerHTML = ""`.

FASE 3-2 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
tudo verde
$ git add .
$ git commit -m "Fase 3-2: busca que filtra a vitrine"
$ git push
```

CONFIRA

Digitar filtra, apagar traz tudo de volta, e o console continua limpo nos cinco passos da conferência.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-20.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-20`



FASE 3-3 · PRÁTICA

O formulário reage

Validar o cadastro antes de enviar — e o produto novo aparecer na vitrine.

FASE 3-3 · NO EDITOR

front/app.js: o formulário de cadastro

front/app.js

```
const form = document.querySelector("#form-produto");
const aviso = document.querySelector("#aviso");

form.addEventListener("submit", (e) => {
  e.preventDefault();

  const nome  = document.querySelector("#nome").value.trim();
  const preco = Number(document.querySelector("#preco").value);

  if (nome === "") {
    aviso.textContent = "Informe o nome do produto.";
    return;
  }
  if (!(preco > 0)) {
    aviso.textContent = "O preço precisa ser maior que zero.";
    return;
  }

  produtos.push({ nome, preco, tam: "M" });
  desenhar(produtos);
  form.reset();
  aviso.textContent = "Produto cadastrado.";
});
```

- `preventDefault` primeiro: sem ele a página recarrega e nada do resto roda.
- `.value` sempre devolve **texto**. `Number(...)` converte; `trim()` tira os espaços.
- `!(preco > 0)` pega o vazio, o zero, o negativo e o `NaN` de uma vez.
- `{ nome, preco }` é atalho para `{ nome: nome, preco: preco }`.
- A mensagem vai para um elemento na página — **nunca** para `alert`.

FASE 3-3 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
tudo verde
$ git add .
$ git commit -m "Fase 3-3: formulário com validação"
$ git push
```

CONFIRA

Enviar vazio mostra o aviso e **não recarrega** a página; enviar preenchido faz o produto aparecer na vitrine na hora.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-21.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-21`

LABORATÓRIO DA EQUIPE

Mãos à obra



- Criar `front/app.js` e ligá-lo no fim do `<body>`.
- Desenhar a listagem do recurso principal a partir de uma lista no próprio arquivo.
- Um campo de busca que filtra com o evento `input`.
- O formulário de cadastro validando no `submit`, com `preventDefault`.
- Console limpo, `pytest` verde, commit e push.



O MAPA DE HOJE

Mapa completo

Nove fases fechadas: a linguagem, o DOM, os eventos — e a vitrine da loja desenhada, filtrada e alimentada por um formulário.

FECHAMENTO

O que cada fase construiu

FASE	CONCEITO	O QUE PROVA
1-1	onde o JS roda; `let` e `const`	dá erro ou não dá
1-2	`===`, os seis valores falsos, função de seta	o `if` entra ou não entra
1-3	array, objeto, `map`, `filter`, `sort`	a tradução do Python
2-1	o DOM como árvore de objetos	os seis passos do arquivo ao comportamento
2-2	`querySelector`, `textContent`, `classList`, `createElement`	por que deu `null`
2-3	`addEventListener`; click, input, submit, `preventDefault`	qual evento usar
3-1	a vitrine desenhada por código	os três produtos na tela, sem HTML escrito à mão
3-2	filtro com `input` e `filter`	os cinco passos da conferência
3-3	validação no `submit`	o aviso aparece e a página não recarrega

ONDE ESTAMOS AGORA

A tela responde. Falta ela conversar com a API.

- Hoje os produtos estavam **escritos dentro do app.js**: cadastrar some quando a página recarrega.
- Na **Aula 13, em 10/11**, entra o `fetch` com `async` e `await`: a vitrine busca os produtos na API de vocês e o formulário grava no banco pelo `POST`.
- A partir dali o projeto está fechado de ponta a ponta — e sobra tempo para a documentação do **Projeto Final, entrega em 17/11**.

PARA ESTUDAR

Referências

- **Trilha JavaScript · Parte A** — sete aulas curtas refazendo o mundo 1 no seu ritmo, sempre partindo do Python. O link fica no mural da turma.
- **MDN Web Docs**, em português — a referência da web: `developer.mozilla.org/pt-BR`
- **javascript.info** — o tutorial moderno, do básico ao avançado.
- **Eloquent JavaScript**, de Marijn Haverbeke — gratuito; os capítulos 13 a 15 são DOM e eventos.
- A trilha **não substitui a aula**: ela é para refazer sozinho o que discutimos aqui.
- Leve a trilha junto no projeto final: ela é consulta rápida.

PARA DEIXAR ABERTO

Cola da aula

QUERO…	ESCREVO
encontrar um elemento	`document.querySelector("#id")`
encontrar vários	`document.querySelectorAll(".classe")`
trocar o texto	`el.textContent = "novo"`
ligar ou desligar um estilo	`el.classList.add("x")` / `.remove("x")`
criar um elemento	`document.createElement("article")`
inserir na página	`pai.append(filho)`
limpar antes de desenhar	`pai.innerHTML = ""`
reagir ao clique	`el.addEventListener("click", fn)`
reagir à digitação	`el.addEventListener("input", fn)`
segurar o envio do formulário	`e.preventDefault()`
ler o que foi digitado	`campo.value`
filtrar uma lista	`lista.filter(x => condição)`
transformar uma lista	`lista.map(x => novo)`
ordenar por número	`[...lista].sort((a, b) => a.n - b.n)`